JALA Academy
Assignment 8

Encapsulation & Data Control

Python solution reference • Code examples are organized by the assignment questions.

How to use this document

Save the code examples into a .py file and run with Python 3. Where a program asks for input, enter values when prompted. For file-handling exercises, create the named text file in the same folder first.

1–4. Student, validation, properties and read-only ID

```
class Student:
    def __init__(self, name="", age=1):
        self.name = name
        self._age = age

    def set_values(self, name, age):
        if age <= 0:
            print("Error: age must be greater than 0")
            return
        self.name = name
        self._age = age

    def get_values(self):
        return self.name, self._age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value <= 0:
            raise ValueError("Age must be greater than 0")
        self._age = value

student = Student("Anu", 20)
student.set_values("Ravi", 22)
print(student.get_values())
student.age = 23
print("Age:", student.age)

class Employee:
    def __init__(self, employee_id):
        self._id = employee_id

    @property
    def id(self):
        return self._id

employee = Employee("E101")
print("Read-only employee ID:", employee.id)
# employee.id = "E102"  # Raises AttributeError because there is no setter.
```

5–8. Bank account, internal variables, computed area, password validation

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        if balance < 0:
            raise ValueError("Initial balance cannot be negative")
        self.account_holder = account_holder
        self._balance = balance

    @property
    def balance(self):
        return self._balance
```

```
    def deposit(self, amount):
        if amount <= 0:
            print("Deposit must be positive")
            return
        self._balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            print("Withdrawal must be positive")
        elif amount > self._balance:
            print("Insufficient balance")
        else:
            self._balance -= amount

account = BankAccount("Anu", 1000)
account.deposit(250)
account.withdraw(100)
print("Balance:", account.balance)

class Example:
    def __init__(self):
        self._internal_value = "Use internally by convention"
example = Example()
print(example._internal_value)  # Accessible, but underscore signals internal use.

class Rectangle:
    def __init__(self, length, width):
        self.length, self.width = length, width
    @property
    def area(self):
        return self.length * self.width
print("Rectangle area:", Rectangle(5, 3).area)

class Password:
    def __init__(self, value):
        if len(value) < 8 or not any(ch.isdigit() for ch in value):
            raise ValueError("Password must be at least 8 characters and contain a number")
        self.value = value
print("Password accepted:", Password("python123").value)
```